Ṛ MACHINE LEARNING
DAY 2 CHEAT SHEET

Classification · Model Evaluation · Unsupervised Learning

ϕ TOPICS COVERED

Regression vs Classification	Model Evaluation Metrics
Logistic Regression	Overfitting vs Underfitting
K-Nearest Neighbors (KNN)	Intro to Unsupervised Learning
Support Vector Machine (SVM)	K-Means Clustering

Each section: Why Use · When to Use · When NOT to Use · Alternatives · Key Parameters

SECTION 1: REGRESSION vs CLASSIFICATION

๑ REGRESSION	⌛ CLASSIFICATION
Output: Continuous number	Output: Discrete category/label
Predicts 'How much?' or 'How many?'	Predicts 'Which class?' or 'Yes/No?'
Examples: Price, Temperature, Age	Examples: Spam/Not Spam, Disease/No Disease
Algorithms: Linear Reg, Ridge, Lasso	Algorithms: Logistic Reg, KNN, SVM, Trees
Loss: MSE, MAE, RMSE	Loss: Cross-Entropy, Log Loss
Eval Metric: R^2 Score	Eval Metric: Accuracy, F1, AUC

Ỏ Why Classification is Needed

Real-world decisions are often binary or categorical — not numerical. When you need to assign a label to an input, classification is the right tool.

Real-World Problem	Type	Output
Email: Spam or Not?	Binary	0 or 1
Cat, Dog, or Bird?	Multi-class	0, 1, 2
Predict House Price	Regression	$250,000
Tumor Malignant?	Binary	Yes / No

SECTION 2: LOGISTIC REGRESSION

WHAT IS IT?

Core Idea	Uses the Sigmoid function to squash any number into a probability between 0 and 1. If probability > 0.5 → Class 1, else → Class 0.
Sigmoid Formula	**σ(x) = 1 / (1 + e^(−x))** → **Always outputs between 0 and 1**
Intuition	Think of it as: 'How confident am I (in %) that this belongs to class 1?' It draws a linear decision boundary in feature space.

USE WHEN	AVOID WHEN	ALTERNATIVES
• Binary classification • Linearly separable data • Need probability output • Need interpretability • Quick baseline model • Large dataset, low compute	• Non-linear boundaries • Complex feature interactions • Heavy class imbalance • Raw text / image data • Missing values present	• SVM (non-linear data) • Decision Tree (non-linear) • Random Forest (complex) • Neural Network (deep) • KNN (irregular boundary)

Key Parameters

Parameter	Default	What it Does / Tip
C	1.0	Regularization strength. Smaller C = stronger regularization (less overfitting). Try: 0.01, 0.1, 1, 10.
penalty	'l2'	Regularization type. 'l1' = sparsity (feature selection), 'l2' = shrink all weights, 'none'.
solver	'lbfgs'	Optimizer. Use 'liblinear' for small datasets, 'saga' for l1 + large data, 'lbfgs' default.
max_iter	100	Increase if model doesn't converge. Try 200–1000.
class_weight	None	Set 'balanced' for imbalanced classes. Automatically adjusts weights.

BEST PRACTICES

Scale Features	ALWAYS use StandardScaler or MinMaxScaler. Logistic Regression is sensitive to feature scale because it uses gradient descent.
Handle Outliers	Remove or cap extreme outliers. They can heavily skew the decision boundary.
Check Multicollinearity	Highly correlated features destabilize coefficients. Use VIF or correlation matrix.
Imbalanced Classes	Use class_weight='balanced' or SMOTE oversampling. Don't just rely on accuracy.

SECTION 3: K-NEAREST NEIGHBORS (KNN)

WHAT IS IT?

Core Idea	No training phase! To classify a new point, find the K closest training points (neighbors) using distance (usually Euclidean), then take a majority vote.
Key Formula	**Euclidean Distance = $\sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$ → Smaller distance = more similar**
K=1 vs K=Large	K=1: Very sensitive, memorizes data (overfitting). Large K: Smoother boundary but may underfit. Best K = √n (rough rule).

USE WHEN	AVOID WHEN	ALTERNATIVES
• Irregular decision boundary • Small datasets (<10K) • No training time needed • Recommendation systems • Anomaly detection (basic)	• Large datasets (slow O(n)) • High-dimensional data (curse) • Noisy / irrelevant features • Real-time predictions needed • Imbalanced classes	• SVM (high-dim, small data) • Decision Tree (non-linear) • Random Forest (large data) • Annoy / FAISS (fast NN)

Key Parameters

Parameter	Default	What it Does / Tip
n_neighbors (K)	5	Number of neighbors to vote. Best K ≈ √n. Odd K avoids ties in binary. Use cross-validation to tune.
metric	'minkowski'	Distance function. 'euclidean' (p=2) is default. Use 'manhattan' (p=1) for sparse data.
weights	'uniform'	'distance' gives closer neighbors more influence. Often improves accuracy.
algorithm	'auto'	'ball_tree' or 'kd_tree' faster for large datasets. 'brute' for small/high-dim.
p	2	Power for Minkowski metric. p=1 → Manhattan, p=2 → Euclidean.

BEST PRACTICES

Normalize Features	CRITICAL! KNN is purely distance-based. Feature with large scale dominates. Use StandardScaler or MinMaxScaler.
Choose K Wisely	Use cross-validation. Plot accuracy vs K. Start with K = √(n_samples). Use odd K for binary to avoid ties.
Reduce Dimensions	High dims = 'curse of dimensionality'. Use PCA or feature selection to reduce noise before KNN.

SECTION 4: SUPPORT VECTOR MACHINE (SVM)

WHAT IS IT?	
Core Idea	Finds the optimal hyperplane that maximally separates two classes. The 'support vectors' are the closest data points to the boundary — they define the margin.
Kernel Trick	Maps data to a higher-dimensional space where it becomes linearly separable. RBF Kernel handles most non-linear cases.
Intuition	Think of it as finding the widest possible road (margin) between two classes. Points on the road edges are the support vectors.

USE WHEN	AVOID WHEN	ALTERNATIVES
• High-dimensional data • Text classification (NLP) • Small-to-medium datasets • Non-linear data (RBF kernel) • Image classification (HOG)	• Very large datasets (slow) • Need probability output • Need interpretability • Data has lots of noise/overlap	• Random Forest (large data) • Gradient Boosting (XGBoost) • Neural Net (very complex) • Logistic Reg (linear, fast)

Key Parameters

Parameter	Default	What it Does / Tip
C	1.0	Regularization. High C = fit training data closely (risk overfit). Low C = wider margin, more errors allowed.
kernel	'rbf'	Kernel type. 'linear' for linearly separable. 'rbf' for non-linear. 'poly' for polynomial boundaries.
gamma	'scale'	RBF kernel coefficient. 'scale'=1/(n_features × X.var()). High gamma = complex boundary (overfit risk).
degree	3	Polynomial kernel degree. Only relevant when kernel='poly'.

SECTION 5: MODEL EVALUATION

⚠ WHY ACCURACY ALONE ISN'T ENOUGH

The Problem	**Imagine 95% of emails are NOT spam. A model that predicts 'not spam' every single time gets 95% accuracy — but is completely useless!**
The Solution	Use Confusion Matrix, Precision, Recall, and F1-Score together. Each metric reveals a different aspect of model performance.

📊 Confusion Matrix

	Predicted: NO	Predicted: YES	
Actual: NO	**TN** True Negative	**FP** False Positive	*← Type I Error*
Actual: YES	**FN** False Negative	**TP** True Positive	*← Type II Error*

📏 Metrics Cheat Sheet

Metric	Formula	What it Measures	Use When
Accuracy	(TP+TN) / Total	Overall correctness	Classes are balanced
Precision	TP / (TP+FP)	Quality of positive predictions	False positives are costly (Spam)
Recall	TP / (TP+FN)	Coverage of actual positives	False negatives are costly (Cancer)
F1-Score	2×(P×R)/(P+R)	Balance of precision & recall	Imbalanced classes
AUC-ROC	Area under ROC curve	Model discrimination ability	Binary classifiers comparison

SECTION 6: OVERFITTING vs UNDERFITTING

OVERFITTING	JUST RIGHT	UNDERFITTING
High Train, Low Test acc Model memorized training data Too complex (too many features) *Analogy: Student memorizes answers without understanding*	High Train ≈ High Test acc Model generalizes well Good bias-variance balance *Analogy: Student understands concepts and applies them*	Low Train AND Low Test acc Model is too simple Didn't learn the pattern *Analogy: Student didn't study at all*

Fix Overfitting	Fix Underfitting
Reduce model complexity	Add more features / complexity
Add more training data	Reduce regularization strength
Use regularization (L1/L2)	Use a more powerful model
Use Dropout (Neural Networks)	Train longer (more epochs)
Feature selection / PCA	Remove noise from data
Cross-validation	Add polynomial features
Early stopping	Ensemble methods

HOW TO DETECT

Learning Curve	Plot train vs validation accuracy over training size. Overfitting: big gap. Underfitting: both curves low.
Validation Loss	If validation loss increases while train loss decreases → overfitting. Both high → underfitting.
Cross-Validation	Use k-Fold CV. High variance across folds → overfitting. Consistently low scores → underfitting.

SECTION 7: INTRO TO UNSUPERVISED LEARNING

1 SUPERVISED LEARNING	‘ UNSUPERVISED LEARNING
Has labeled data (X, y)	No labels — only X
Learn a mapping from X → y	Find hidden structure/patterns
Goal: Predict	Goal: Discover
Examples: Classification, Regression	Examples: Clustering, Dimensionality Reduction
Eval: Accuracy, MSE, F1	Eval: Silhouette Score, Inertia
Algorithms: Logistic Reg, SVM, KNN	Algorithms: K-Means, PCA, DBSCAN

‘ Key Use Cases for Unsupervised Learning

Customer Segmentation	Anomaly Detection	Data Compression	Recommendation
Group users by behaviour without predefined labels	Find unusual patterns in financial fraud or network intrusion	PCA reduces 100 features to 2 for visualization	Find similar products/movies for collaborative filtering

SECTION 8: K-MEANS CLUSTERING

WHAT IS IT?	
Core Idea	Partition data into K clusters by iteratively assigning each point to its nearest centroid and updating centroids until stable.
Algorithm Steps	1. Choose K (number of clusters) 2. Randomly initialize K centroids 3. Assign each point to nearest centroid 4. Update centroids (mean of cluster points) 5. Repeat 3-4 until convergence (no change)
Elbow Method	Plot Inertia (WCSS) vs K. The 'elbow' point — where adding more K gives diminishing returns — is the optimal K.

USE WHEN	AVOID WHEN	ALTERNATIVES
• Customer segmentation • Image compression • Document clustering • Spherical, compact clusters • K is roughly known	• Non-spherical clusters • Clusters of varying density • K is completely unknown • Categorical features only • Many outliers present	• DBSCAN (arbitrary shapes) • Hierarchical Clustering • Gaussian Mixture Models • Mean Shift (auto K)

Key Parameters

Parameter	Default	What it Does / Tip
n_clusters (K)	8	Number of clusters. Use Elbow Method or Silhouette Score to find optimal K.
init	'k-means++'	Initialization method. 'k-means++' is smarter than random, leads to faster convergence.
n_init	10	Number of times to run with different seeds. Takes the best result. Increase for stability.
max_iter	300	Max iterations per run. Increase if not converging.
random_state	None	Set a seed (e.g. 42) for reproducible results.

BEST PRACTICES	
Scale Features	K-Means uses Euclidean distance. Features on different scales will dominate. ALWAYS use StandardScaler first.
Elbow Method	Try K=2 to 10. Plot inertia (WCSS). Optimal K is where the curve bends like an elbow. Silhouette score > 0.5 is good.
Multiple Runs	K-Means is sensitive to initialization. Set n_init=10+ and use init='k-means++' to avoid bad local minima.
Evaluate	Use Silhouette Score (-1 to 1, higher=better), Davies-Bouldin Index (lower=better), or visual inspection.

SECTION 9: HANDS-ON CODE REFERENCE

ʒ Setup & Imports

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import numpy as np, pandas as pd
```

ȷ Preprocessing Template

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Scale features (ALWAYS do this before LR, KNN, SVM)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # fit + transform on train
X_test = scaler.transform(X_test)        # only transform on test
```

Ṛ Train & Evaluate Classifiers

```
# ── Logistic Regression ──────────────────────────────
lr = LogisticRegression(C=1.0, max_iter=200, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print('LR Accuracy:', accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))

# ── KNN ──────────────────────────────────────────────
knn = KNeighborsClassifier(n_neighbors=5, weights='distance')
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
print('KNN Accuracy:', accuracy_score(y_test, y_pred_knn))

# ── SVM ──────────────────────────────────────────────
svm = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print('SVM Accuracy:', accuracy_score(y_test, y_pred_svm))

# ── Confusion Matrix ─────────────────────────────────
print(confusion_matrix(y_test, y_pred_lr))

# ── Cross-Validation ─────────────────────────────────
scores = cross_val_score(lr, X, y, cv=5, scoring='f1_macro')
print(f'CV F1: {scores.mean():.3f} +/- {scores.std():.3f}')
```

ɷ K-Means Clustering + Elbow Method

```
# Scale first!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
# ── Elbow Method ──────────────────────────────────────────
inertias = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(K_range, inertias, 'bo-')
plt.xlabel('K'); plt.ylabel('Inertia'); plt.title('Elbow Method')
plt.show()

# ── Fit K-Means with optimal K ────────────────────────────
km = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)

# ── Evaluate ──────────────────────────────────────────────
score = silhouette_score(X_scaled, labels)
print(f'Silhouette Score: {score:.3f}')  # > 0.5 is good
print('Inertia (WCSS):', km.inertia_)
```

QUICK REFERENCE: ALGORITHM COMPARISON

Algorithm	Type	Training	Prediction	Interpretable?	Scaling Needed?	Best For
Logistic Reg	Supervised	Fast	Fast	Yes	Yes	Linear binary
KNN	Supervised	None	Slow O(n)	Yes	Yes	Small, irregular
SVM (linear)	Supervised	Medium	Fast	Partial	Yes	High-dim data
SVM (RBF)	Supervised	Slow	Medium	No	Yes	Non-linear data
K-Means	Unsupervised	Medium	Fast	Yes	Yes	Clustering
DBSCAN	Unsupervised	Medium	Medium	Partial	Yes	Arbitrary shapes